INDRA

India's National Disaster Risk Analysis Platform

Session report: data pipeline, ML hazard classifier, and dashboard connection

Date	29 October 2026
Repository	github.com/Benjamin-G-Leo/INDRA_SIH (branch main)
New code lives in	weather_platform_v2/ (the original weather_platform/ is kept unchanged)
Prepared with	Claude Code

1. Summary

INDRA is meant to detect weather disasters, verify them using public posts and media, and alert emergency services. No IMD or social-media API keys are available yet, so the prototype runs on synthetic (randomly generated) data. This session reviewed the whole project, found and fixed the weak spots in the data pipeline, made the synthetic data much more realistic, and built a small machine-learning model that labels the hazard type of a social-media post.

- **Reviewed** both parts of the project: the Python data pipeline and the Next.js dashboard. Ran tests and built both.
- **Fixed** repo clutter and a misleading evaluation metric.
- **Improved** the data generator so it contains the hard cases real posts have.
- **Built** a TF-IDF + logistic regression classifier. It labels 97% of 113 hand-written posts correctly on its own, 99% combined with the rules.
- **Organised** all of this session's work into a new folder, and committed and pushed it to GitHub.
- **Connected** the teammate's new dashboard (dashboard_prototype_2) to the pipeline, fixed eight dashboard bugs, and checked it running in a browser.

2. What the project is

The repository has two parts that are not yet connected:

Part	What it does	State at start
weather_platform/ (Python)	Generates 2,000 fake IMD bulletins and 3,000 fake social posts, then cleans them: parsing, unit conversion, spam and irrelevance filters, duplicate removal, rule-based hazard tagging, credibility scoring, and a train/val/test split by event. Stores everything in SQLite or MySQL.	Works. 15 of 15 tests pass.
dashboard_prototype_1/ (Next.js)	Map, KPI cards and alert list for disaster incidents.	Builds and type-checks cleanly, but all data is hard-coded in lib/data.ts.

3. Review findings at the start

Nothing crashed. The issues found were:

#	Finding	Impact
1	weather_platform/weather_platform/ was an exact copy of its parent folder.	Clutter, confusion
2	A .db file and generated CSVs were committed; there was no root .gitignore.	Repo bloat
3	The evaluation printed 'genuine records wrongly rejected: 40'. About 33 of those were resent IMD bulletins where the pipeline kept the copy and dropped the original. The identical rows made this harmless, so the number was misleading.	Misleading metric
4	No machine-learning model existed, although the data is prepared for one (hazard_label and data_split columns).	Missing feature
5	The synthetic posts came from a few fixed templates, so any model would score near 99% and prove nothing.	Unrealistic data
6	Dashboard footer says 'PostgreSQL data layer' but no database or API exists; package name is 'my-project'; two lock files (npm and pnpm).	Cosmetic / prototype
7	Known open items: MySQL path never tested; duplicate detection only works within one batch; corrected IMD bulletins are not applied as updates.	Future work

4. Work done

4.1 Cleanup and evaluation fix

- Removed the nested duplicate folder and added a root `.gitignore` (virtual environments, caches, .db files, models).
- Duplicates are now scored per original/copy pair using a new hidden column `_gt_dup_of`. Keeping the copy instead of the original is no longer counted as lost data, and the 'genuine lost' count ignores deduplication decisions.

4.2 More realistic synthetic data

The generator (`generate_dataset.py`) now mixes in the kinds of posts a model has to cope with in real life:

Addition	Share / detail	Why it matters
Indirect posts with no hazard keyword (e.g. 'cars floating, need rescue')	About 12% of genuine posts	Rules cannot label these; only ML can
Extra phrasings per hazard (news style, Hinglish, Hindi)	About 25% of genuine posts	Less template repetition
Vague, sarcastic, question, forecast and mixed-hazard posts	About 8%	Realistic ambiguity
SMS short-forms ('pls', 'hvy') and three typo kinds (swap, drop, double)	6% and 8%	Messy text
Weather-word posts where nothing is happening (e.g. 'hope the weather is good for my trip')	8 new templates	Tests the junk filter
Annotator label noise between neighbouring hazards	2% of labels; truth kept in _gt_hazard	Real labels are never perfect

The lexicons (`lexicons.py`) received a few extra impact words and non-event phrases so the relevance filter handles the new hard cases. As a result the rule tagger now covers about 90% of posts instead of 98%, which gives the ML model real work to do.

4.3 ML hazard classifier

`train_classifier.py` reads the cleaned posts from the database, trains on the train split, tunes on validation, and scores on the test split and on a hand-written set. The model combines word patterns (1-2 words) and character patterns (2-5 letters) so typos and Hinglish still match, feeding a logistic regression. It trains in seconds on a laptop without a GPU.

- **Hand-written test set:** `tests/hand_labeled.csv` has 113 posts (about 5 per hazard) in English, Hinglish and Hindi, written separately from the generator.
- **Hybrid use:** keep the rule tag when a rule fires; ask the model only when no rule fires.
- **Reuse:** `predict(load_model(), [text])` returns (hazard, probability) pairs.

5. Results

5.1 Pipeline quality on the new synthetic data (5,000 raw rows)

Stage	Recall	Precision	Note
Spam filter	95%	100%	No genuine post removed as spam
Irrelevant-post filter	100%	90%	Hard 'weather word, no event' posts included
Duplicate removal (pairs)	94%	97%	Per original/copy pair
Rule tagger	90% coverage	98% accuracy	On the posts where a rule fires

Output: 1,934 clean IMD rows, 2,314 clean public posts, 752 rejected. 12 genuine posts were wrongly rejected and 0 noise rows leaked into the clean data. Re-running with three other random seeds and 8,000 rows gave similar numbers and no crashes.

5.2 Classifier accuracy

Test set	Rules only	Model only	Rules + model
Hand-written (113 posts) - the honest score	75%	97%	99%
Synthetic validation split (321 posts)	88%	98%	98%
Synthetic test split (349 posts)	89%	98%	98%

The rules fire on only 76% of the hand-written posts. The model labelled all 27 posts the rules missed correctly. It got 3 of 113 wrong: Hot & Humid read as Warm Night, Extreme Wind read as Heavy Rainfall, and Dust Storm read as Dust-Raising Winds. The first and last are confusions that people make too.

6. Dashboard connection

A teammate pushed an updated dashboard as `dashboard_prototype_2/`. It type-checked, passed its 12 tests and built cleanly, but it ran on a hand-made fixture. This session connected it to the data pipeline.

6.1 How it is connected

The new script `weather_platform_v2/export_dashboard.py` reads the cleaned database and writes `dashboard_prototype_2/public/data/indra-dataset.json` in the schema the teammate defined (version 1.0.0).

Step	What it does
1. Hazard per post	Rule tag if a rule fired, otherwise the ML classifier's guess. Ground-truth columns are never used.
2. Group into incidents	Posts with the same state and hazard are chained into one incident while no gap exceeds 6 hours.

Step	What it does
3. Verify with IMD	An IMD bulletin for the same state and hazard inside the window verifies the incident, sets its severity colour and supplies its reading (for example 48.2 C or 341 mm).
4. Score and select	Confidence is the mean post credibility. The 60 best incidents are kept; alerts are created only where IMD gives a severity.
5. Dashboard	Loads the JSON, validates it, and re-reads it every 10 seconds or on Refresh. If the file is missing it shows an error and never falls back to other data.

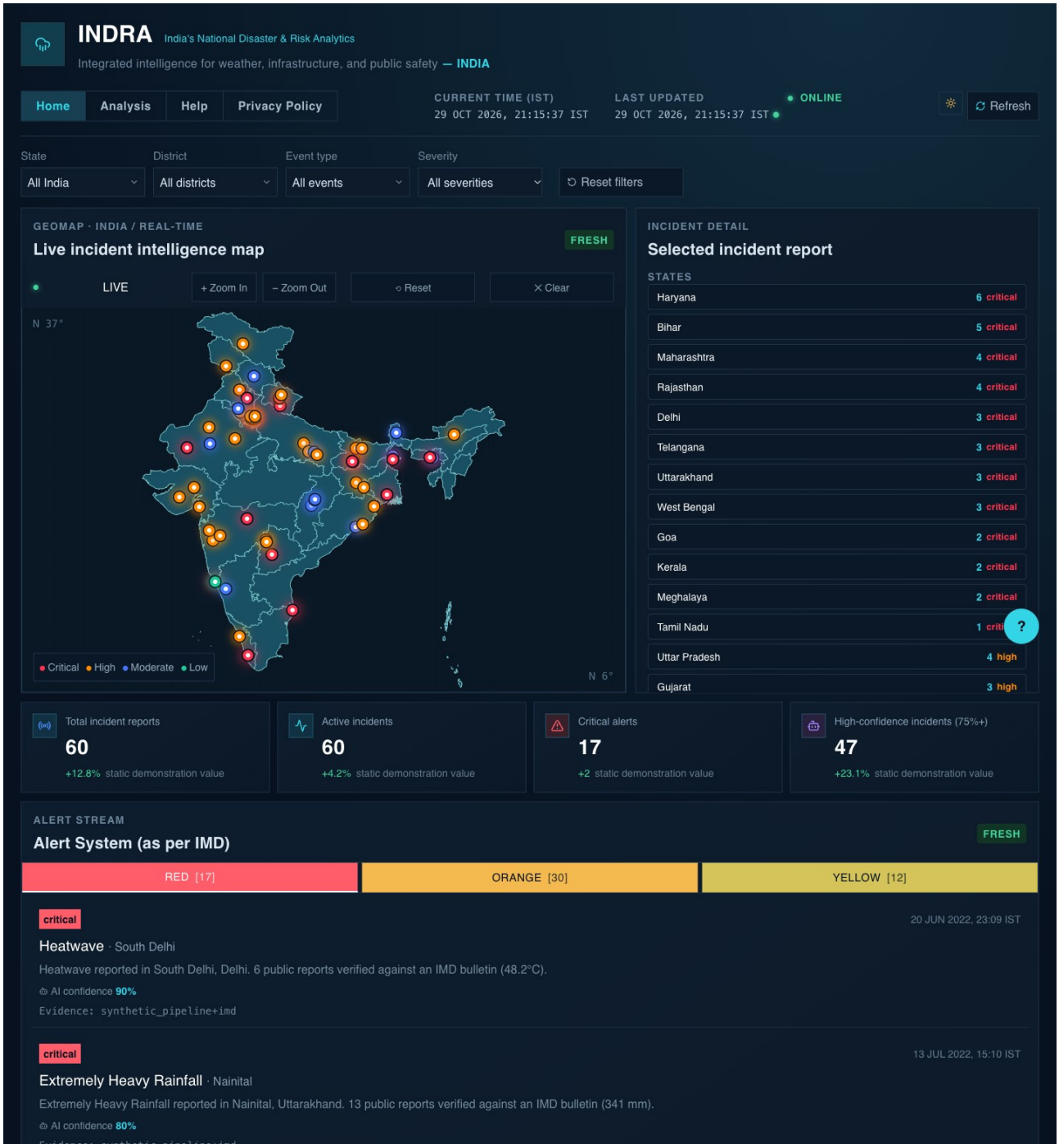
Output: 60 incidents, 59 alerts and 885 source records. Every record is marked synthetic.

6.2 Bugs found and fixed

Bug	Effect	Fix
Filter said 'Hot and Humid', data says 'Hot & Humid'	That hazard could never be filtered	Name corrected
Bar charts added a % sign and scaled bars to 100	5 incidents showed as '5%'	Bars scale to the largest value; no % for counts
10-second 'sync' only waited 400 ms	Nothing was reloaded	Now re-reads the dataset
Time chart counted back from today's date	Chart empty for 2022-2025 data	Window ends at the newest report
A failed refresh replaced the whole screen with an error	Dashboard blanked on a network blip	Keeps showing the last good data
Analysis page had a main element inside another main	Invalid page structure	Inner element changed to a div
Times shown as raw text (2022-06-20T17:39:00Z)	Hard to read	Formatted as 20 JUN 2022, 23:09 IST (with tests)
Analysis panels used invented numbers (84% agreement, 128,450 reports)	Misleading	Computed from the loaded data
Incident with no reading showed a blank 'current reading'; KPI 'AI-detected incidents' showed 0	Confusing	'Not reported' fallback; KPI renamed 'High-confidence incidents (75%+)'

6.3 The running dashboard

Screenshots taken from the production build running locally (npm run build, npm start) at 1440 px width.



Home: map of the 60 incidents (coloured by IMD severity), incident list by state, KPI cards and the start of the alert stream.



Selecting a state and district opens incident cards with severity, IMD reading, confidence and number of reports.



Analysis: reports over time, incidents by type and state, reports by source, and reliability figures, all computed from the pipeline data.

6.4 Dashboard limitations that remain

- The KPI change figures (for example +12.8%) and the System Health panel are still demonstration values; the dashboard labels them as such.
- 'Source consensus: Medium' on each incident is a fixed placeholder from the teammate's adapter, not a measured value.
- All 60 incidents show as Verified because the generated data has an IMD bulletin for nearly every event. Incident dates are 2022-2025.
- dashboard_prototype_2 still tracks about 35 scratch files (build logs, fix scripts, __pycache__). They were left alone; cleaning them up is the teammate's call.

7. Repository changes

Commit	What changed
d42ec5c	Improved dataset realism, fixed evaluation metric, removed duplicate folder
c76f7b9	Added ML hazard classifier and the hand-labelled test set
e42a9ad	Moved this session's work into weather_platform_v2/ and restored weather_platform/ to its original state
548ec50	Connected dashboard_prototype_2 to the pipeline (exporter, dashboard fixes, tests, docs)
d34d346	Removed dashboard_prototype_1, which only held an older nested copy of prototype 2
latest	Reading fallback and KPI wording, regenerated dataset, report updated with the dashboard section

Folder layout: `dashboard_prototype_2/` (dashboard), `weather_platform/` (original version), `weather_platform_v2/` (improved pipeline and ML model), and this `Project Report/` folder.

8. How to run it

From inside `weather_platform_v2/` (dashboard commands at the end):

```
python3 -m venv .venv
source .venv/bin/activate    # Windows: .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python generate_dataset.py
python ingest.py --reset --export data/clean
python train_classifier.py
python export_dashboard.py
python -m unittest discover -s tests -v

# dashboard (new terminal)
cd ../dashboard_prototype_2
npm install
npm run dev    # then open http://localhost:3000
```

9. Limits to be honest about

- **Synthetic data.** Scores on the generator's own splits are optimistic because it reuses templates. In a demo, quote the hand-written figure and say the model is trained on synthetic data.
- **The hand-written set was written by Claude** after reading the generator, so it is less independent than posts from real people. Fifty more posts from a teammate, written without looking at the code, would give a fairer score.
- **The model only picks a hazard.** It does not decide whether a post is real. Junk is removed earlier by the spam and relevance filters, which must run first.
- **Model files are pickles.** Only load model files you trained yourself.
- **Small losses remain.** The filters wrongly rejected 12 genuine posts in 5,000, mostly ones that describe the event without a weather word.

10. Suggested next steps

- Have a teammate write 50 or more extra test posts blind, and re-score the model.
- Add a 'is this report credible?' score that combines the model, corroboration, source trust and the IMD match - the 'verify' step of the pitch.

- Replace the exported JSON file with a live API route over the database, and replace the remaining demonstration values (KPI changes, System Health).
- Test the MySQL path and let duplicate detection look at earlier batches.
- When API access arrives, swap the CSV reader in ingest.py for a live feed and retrain on real posts.